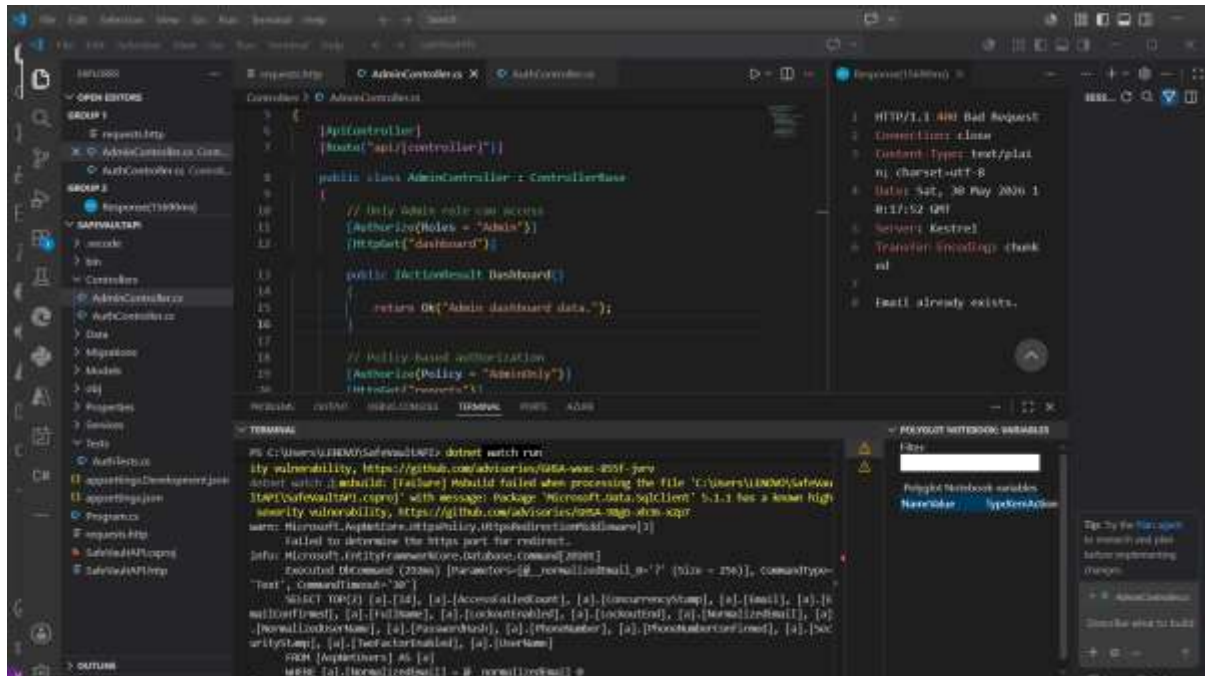
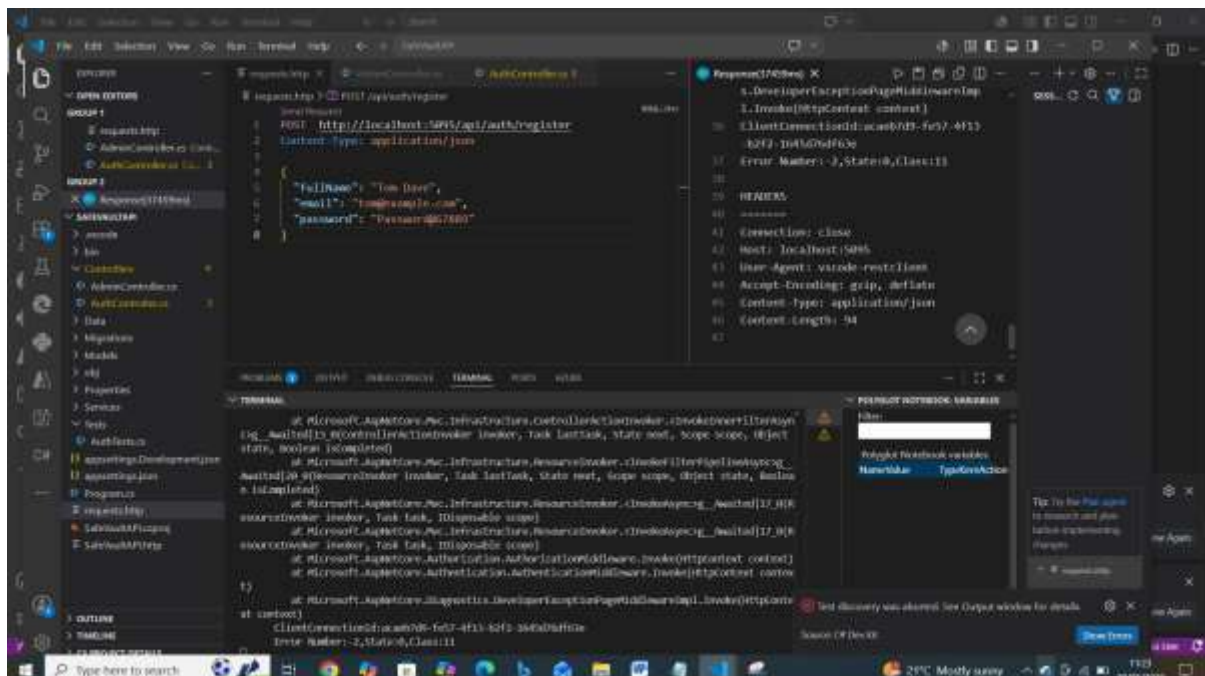


TESTING AND DEBUGGING

Rest client was installed from the visual studio code IDE extension. The application was tested using watch run.

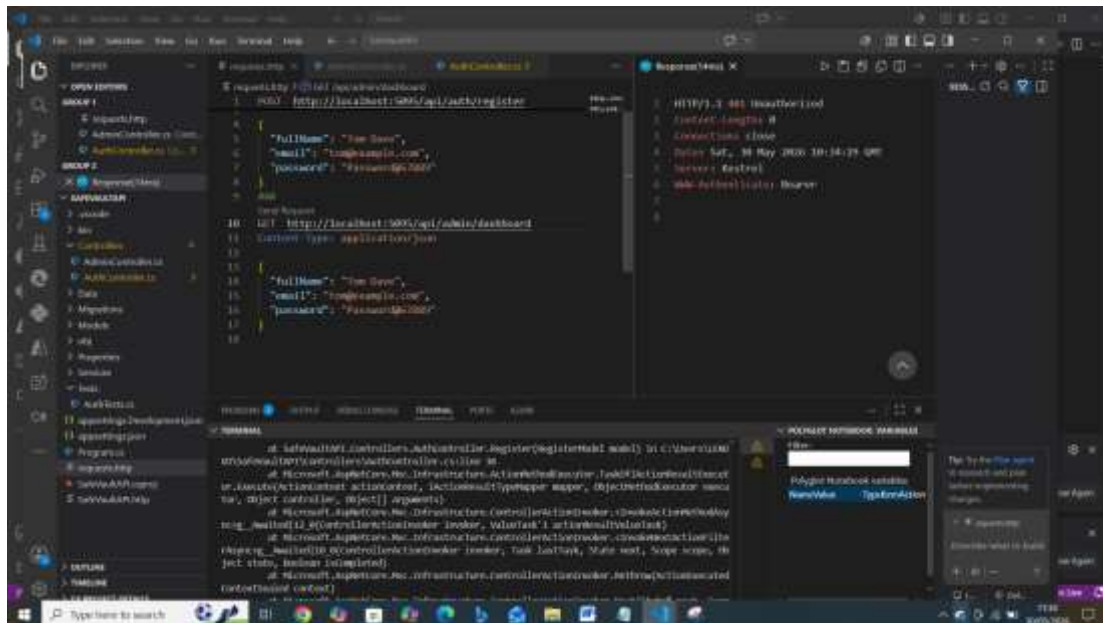


The application runs successfully.



Initially after attempting to register a user the headers were displayed because I have not login.

Kayode Oladipo –Debugging assignment



I made this request in my application:

GET <http://localhost:5095/api/admin/dashboard>

Content-Type: application/json

```
{
  "fullName": "Tom Dave", "email": "tom@example.com", "password":
  "Password@67889"
}
```

This is what I am getting

HTTP/1.1 401 Unauthorized Content-Length: 0 Connection: close Date:
Sat, 30 May 2026 10:34:29 GMT Server: Kestrel WWW-Authenticate:
Bearer.

This demonstrates that the data I stored in the register is protected and needs authorisation may be a bearer 's token to allow me to access the data.

The response:

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer

my endpoint is protected by JWT Bearer authentication, and my request does not contain a valid access token. The dashboard endpoint that is a JWT token. I know this is the case, so I must

Kayode Oladipo –Debugging assignment

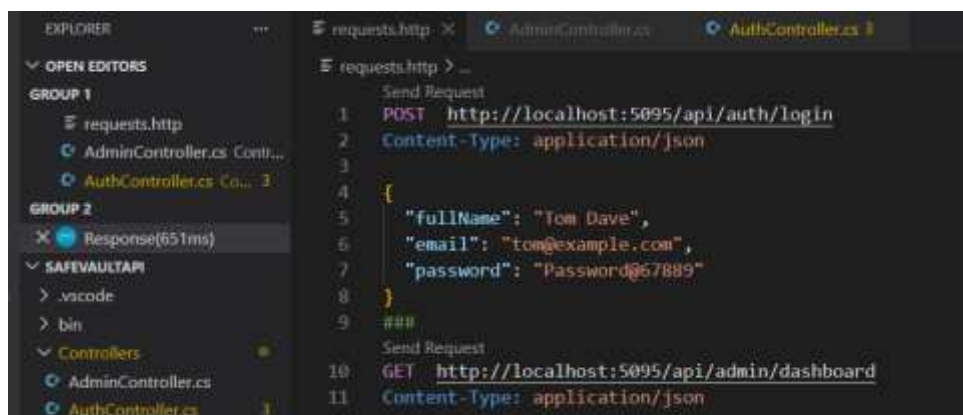
first log in and obtain a token. Then call the dashboard endpoint with:

```
GET http://localhost:5095/api/admin/dashboard
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

The error is not related to authorization itself. Your JWT token is failing to be generated, because your secret key is too short.

The important part is:



Debugging using AI

Another error occurs as shown below:

```
IDX10720: Unable to create KeyedHashAlgorithm for algorithm 'HS256',  
the key size must be greater than: '256' bits, key has '208' bits.
```

What this means that 'HS256' requires a secret key of at least 256 bits (32 bytes).

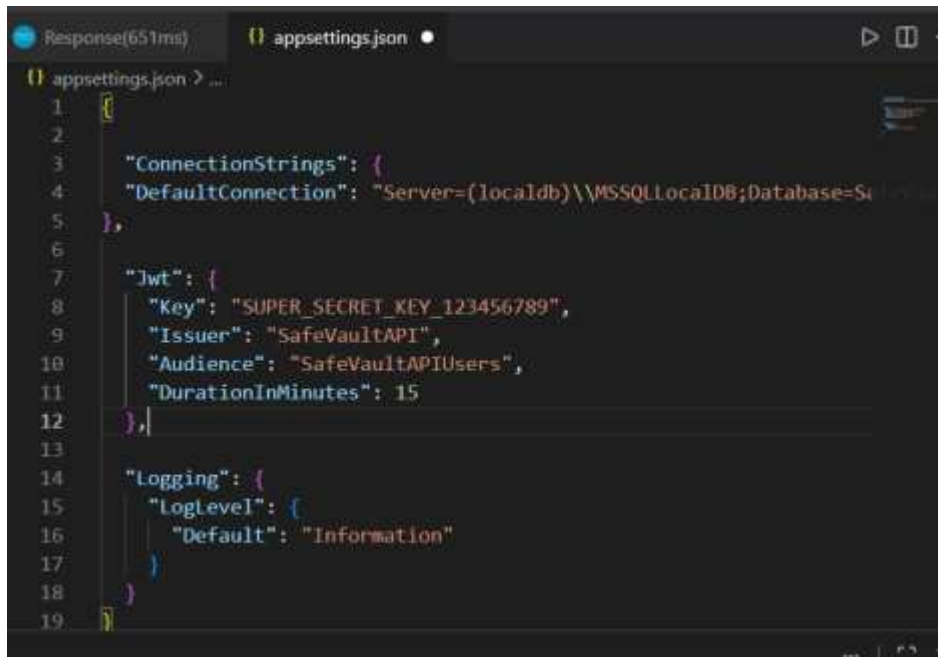
My key is only 208 bits (26 bytes).

Correction

My appsettings.json contains the followings:

```
{  
  
  "Jwt": {  
  
    "Key": "SUPER_SECRET_KEY_123456789",
```

```
}  
  
}
```



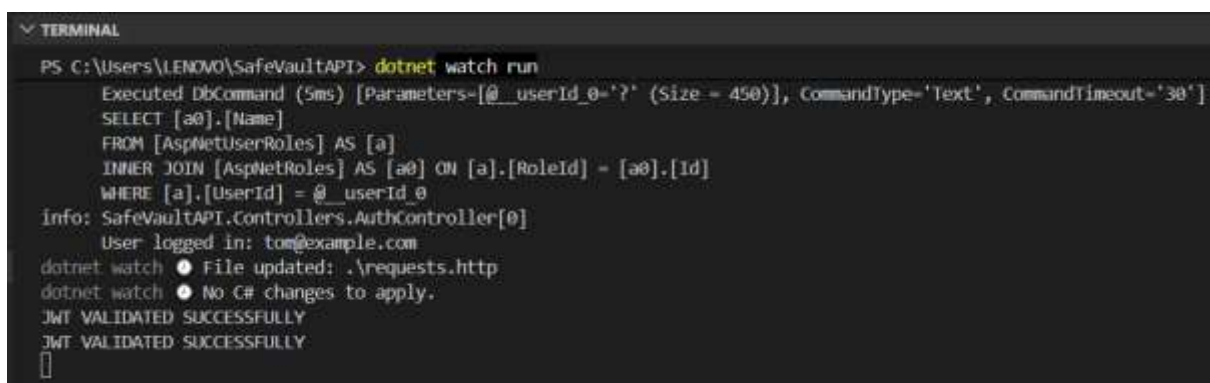
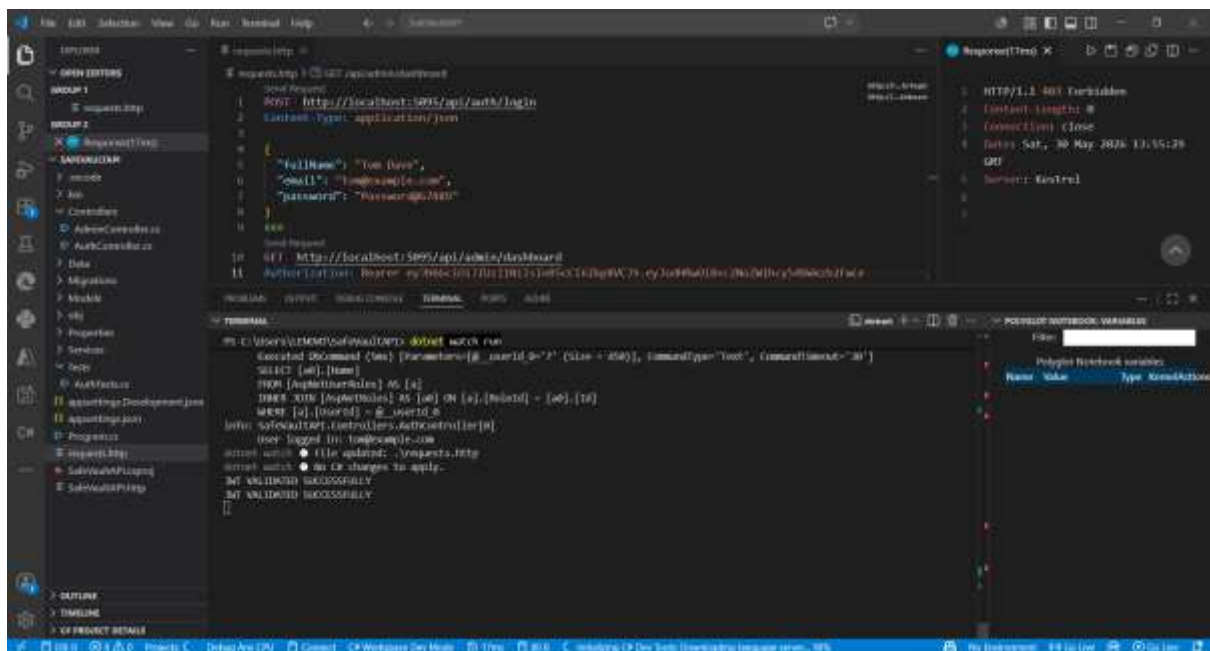
I could also have used the followings in my appsettings.json

```
{  
  
  "Jwt": {  
  
    "Key": "7F4D9A8C2B1E6F3A5D8C9E1B4F7A2D6C9E5F1A8B3D7C4E6F"  
  
  }  
  
}
```

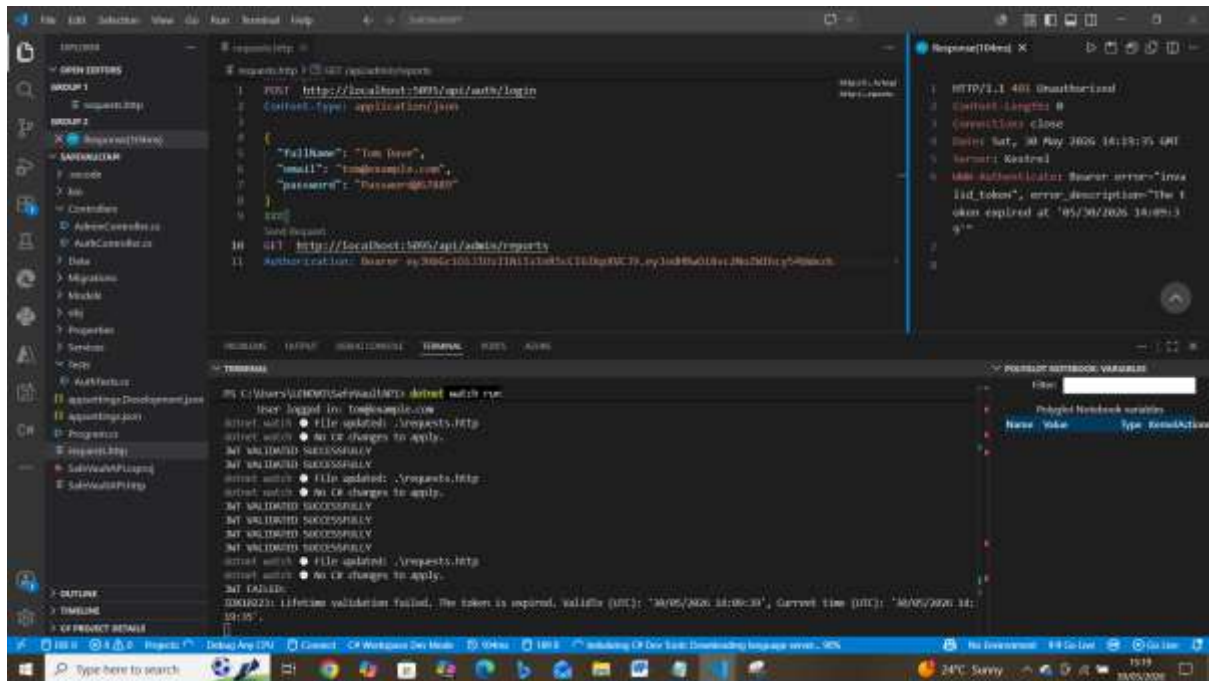
So I corrected it as shown below:

Kayode Oladipo –Debugging assignment

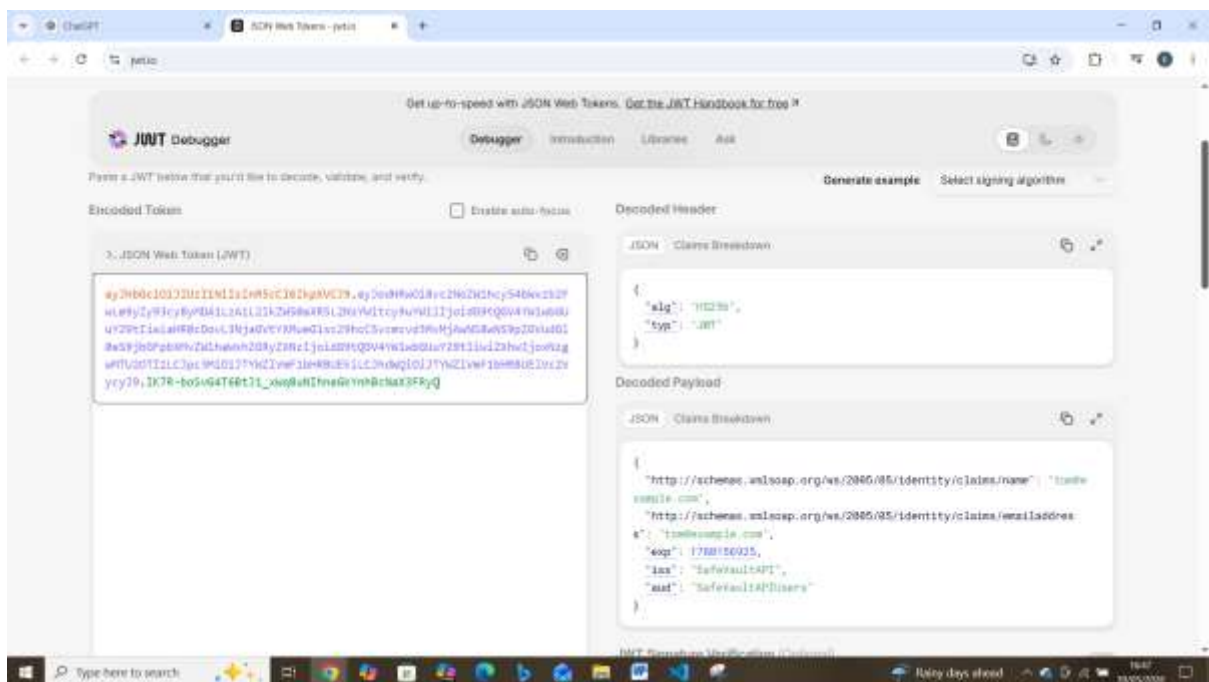
Accessing the dashboard.



Kayode Oladipo –Debugging assignment

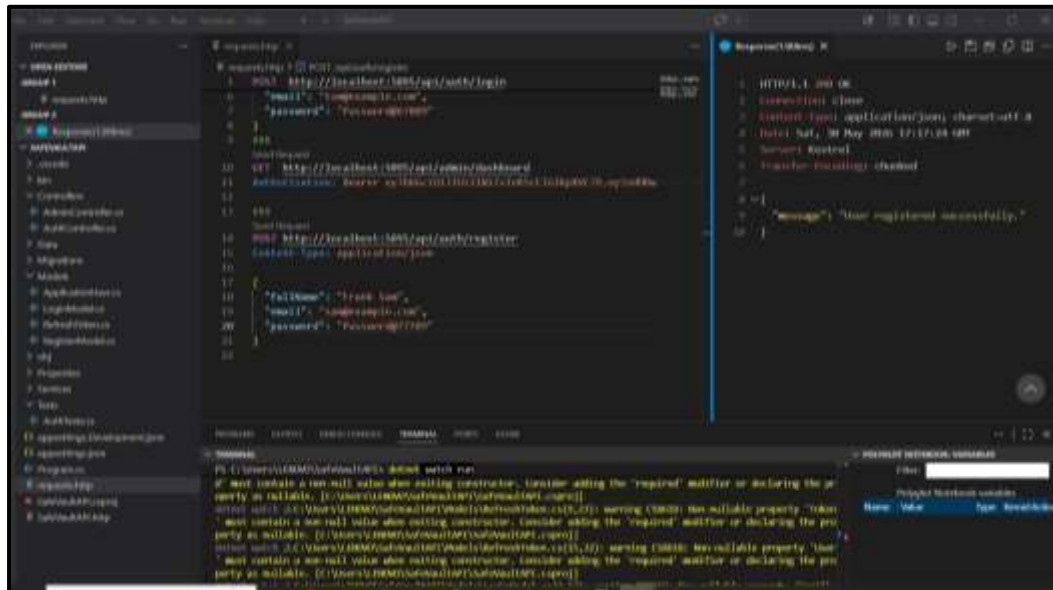


Checking my token to see the payload contains the right data.

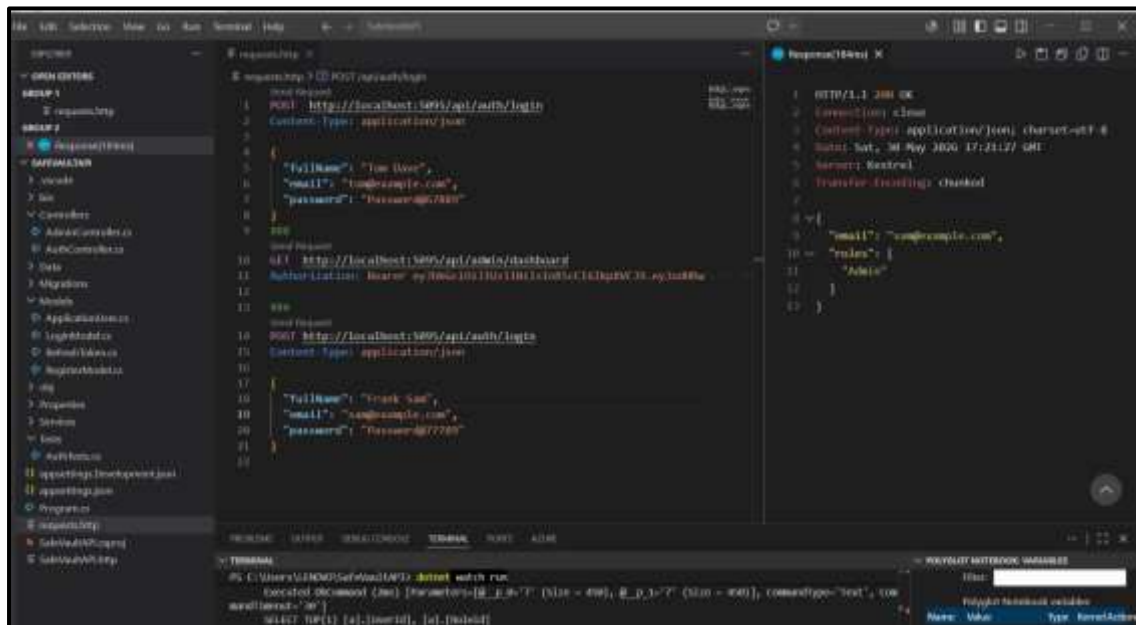


Kayode Oladipo –Debugging assignment

Registering a user.



Login in to make a request.



Kayode Oladipo –Debugging assignment

The screenshot shows the VS Code interface with a REST client file named `requests.http`. The first request is a POST to `http://localhost:5095/api/auth/login` with a JSON body containing user credentials. The response is a 200 OK status with a JSON body containing user details.

```
1 POST http://localhost:5095/api/auth/login
2 Content-Type: application/json
3
4 {
5   "fullName": "Frank Sam",
6   "email": "sam@example.com",
7   "password": "Password@77789"
8 }
9
10
11
12
13
14
15
16
17
18
19
20 GET http://localhost:5095/api/admin/dashboard
21 Content-Type: application/json
22
23 {
24   "fullName": "Frank Sam",
25   "email": "sam@example.com",
26   "password": "Password@77789"
27 }
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
1 HTTP/1.1 200 OK
2 Connection: close
3 Content-Type: application/json; charset=utf-8
4 Date: Sat, 30 May 2020 17:25:43 GMT
5 Server: Kestrel
6 Transfer-Encoding: chunked
7
8 {
9   "email": "sam@example.com",
10  "roles": []
11 }
```

The screenshot shows the VS Code interface with a REST client file named `requests.http`. The first request is a GET to `http://localhost:5095/api/admin/dashboard` with a Bearer token in the Authorization header. The response is a 200 OK status with a JSON body containing user details.

```
1 GET http://localhost:5095/api/admin/dashboard
2 Content-Type: application/json
3 Authorization: Bearer
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

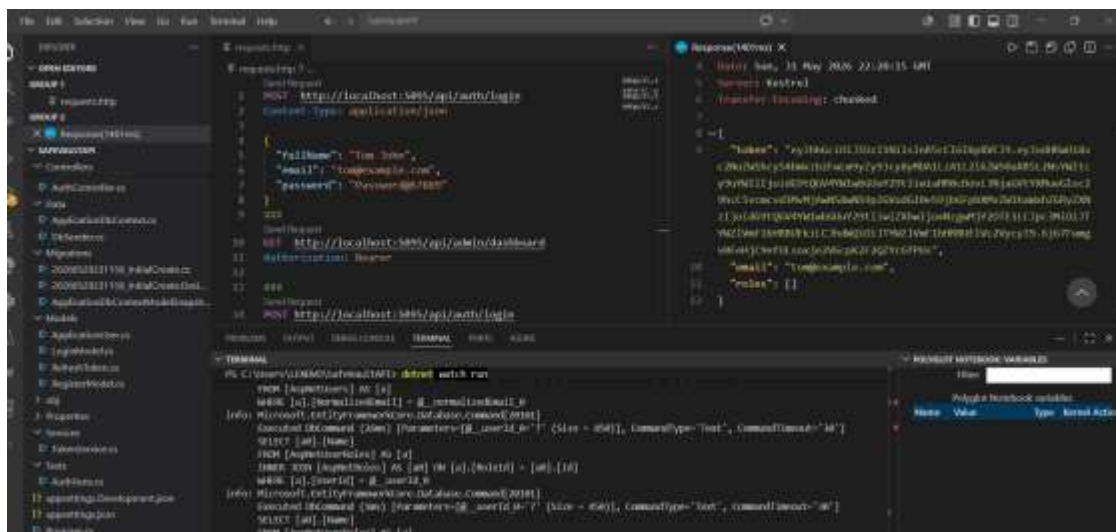
```
1 HTTP/1.1 200 OK
2 Connection: close
3 Content-Type: application/json; charset=utf-8
4 Date: Sat, 30 May 2020 17:25:43 GMT
5 Server: Kestrel
6 Transfer-Encoding: chunked
7
8 {
9   "email": "sam@example.com",
10  "roles": [
11    "Admin"
12  ]
13 }
```

The screenshot shows the VS Code interface with a REST client file named `requests.http`. The first request is a GET to `http://localhost:5095/api/admin/dashboard` with a Bearer token in the Authorization header. The response is a 200 OK status with a JSON body containing user details.

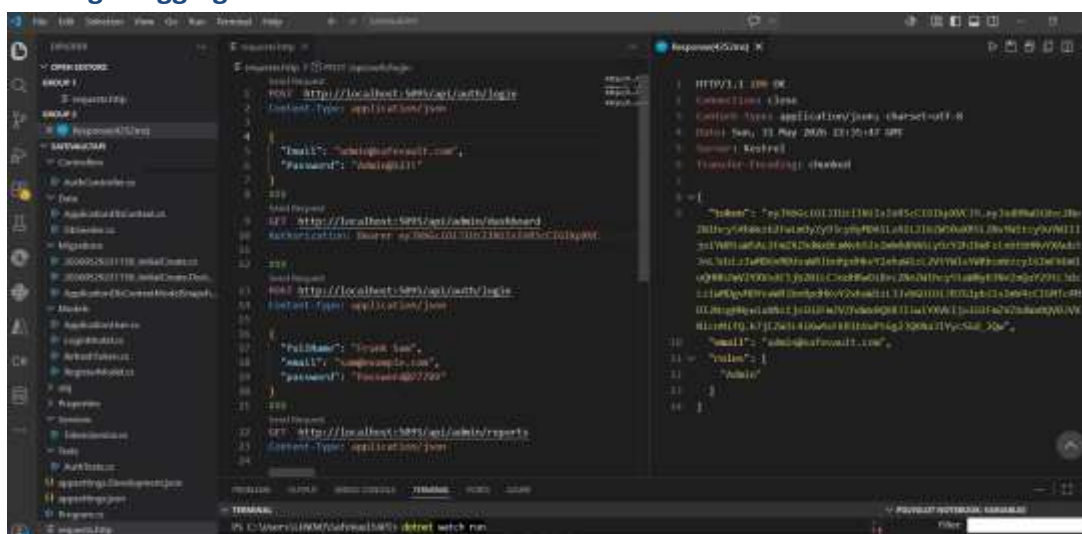
```
1 GET http://localhost:5095/api/admin/dashboard
2 Content-Type: application/json
3 Authorization: Bearer
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
1 HTTP/1.1 200 OK
2 Connection: close
3 Content-Type: application/json; charset=utf-8
4 Date: Sat, 30 May 2020 17:39:16 GMT
5 Server: Kestrel
6 Transfer-Encoding: chunked
7
8 {
9   "email": "sam@example.com",
10  "roles": [
11    "Admin"
12  ]
13 }
```

Logging in as a user



Testing: Logging in as an administrator

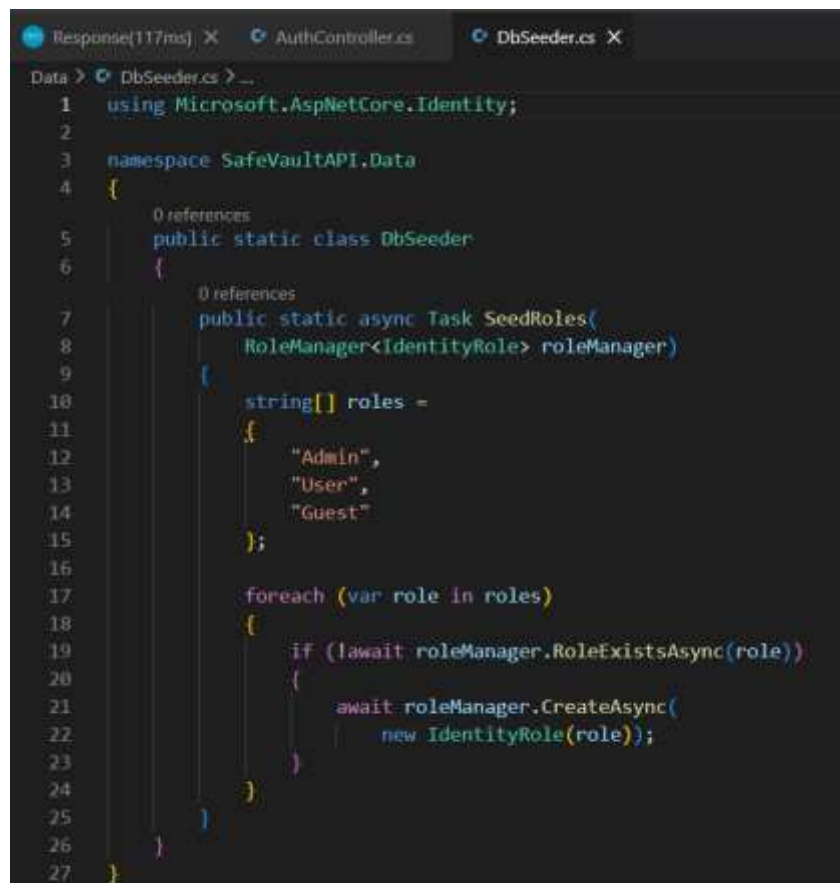


Page 10 of 15


```
113         var roles = await _userManager.GetRolesAsync(user);
114
115     var token = await GenerateJwtToken(user);
116
117     return Ok(new
118     {
119         token,
120         email = user.Email,
121         roles = roles
122     });
123 }
```

DBSEEDER.CS MODIFICATION

The DbSeeder.cs was modified as well as shown below:



```
1 using Microsoft.AspNetCore.Identity;
2
3 namespace SafeVaultAPI.Data
4 {
5     0 references
6     public static class DbSeeder
7     {
8         0 references
9         public static async Task SeedRoles(
10             RoleManager<IdentityRole> roleManager)
11         {
12             string[] roles =
13             {
14                 "Admin",
15                 "User",
16                 "Guest"
17             };
18
19             foreach (var role in roles)
20             {
21                 if (await roleManager.RoleExistsAsync(role))
22                 {
23                     await roleManager.CreateAsync(
24                         new IdentityRole(role));
25                 }
26             }
27 }
```

The Dbseeder.cs complete codes look like the followings:

```
using Microsoft.AspNetCore.Identity;
namespace SafeVaultAPI.Data
{
    public static class DbSeeder
    {
        public static async Task SeedRoles(
            RoleManager<IdentityRole> roleManager)
```

```
{
    string[] roles =
    {
        "Admin",
        "User",
        "Guest"
    };
    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
        {
            await roleManager.CreateAsync(
                new IdentityRole(role));
        }
    }
}
```

Modifying the Program.cs

Currently Program.cs contains:

```
using (var scope = app.Services.CreateScope())
{
    var roleManager =
        scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    await DbSeeder.SeedRoles(roleManager);
}
```

It should contain:

Kayode Oladipo –Debugging assignment

```
using Microsoft.AspNetCore.Identity;

namespace SafeVaultAPI.Data
{
    public static class DbSeeder
    {
        public static async Task SeedRoles(
            RoleManager<IdentityRole> roleManager)
        {
            string[] roles =
            {
                "Admin",
                "User",
                "Guest"
            };

            foreach (var role in roles)
            {
                if (!await roleManager.RoleExistsAsync(role))
                {
                    await roleManager.CreateAsync(
                        new IdentityRole(role));
                }
            }
        }
    }
}
```

Making API request as an admin for admin reports. Again I login as an admin, a jwt is generated and this was used to access admin reports. This can be seen here as sensitive reports

